



Instamint

Report

MSI DEV A

Prepared By:

MARTIN Jimmy, MORGAT Mathieu,
CHAUDHARY Prajay, MEHDAOUI Adam

July 14, 2024

TABLE OF CONTENTS

Planning.....	4
Durability of chosen solutions.....	5
Quality processes.....	6

Planning followed

The project must be delivered by June 14, 2024, which is the middle of Sprint 3. Our planning was structured over three months to ensure we meet this deadline effectively.

Here is the timeline we followed over the three sprints:

- **Sprint 1 (April):** Initial setup and foundation tasks.
- **Sprint 2 (May):** Development and integration of core features.
- **Sprint 3 (June):** Finalization and preparation for delivery, ensuring all functionalities are complete and tested.

By following this structured planning, we ensured that all critical components were addressed timely, allowing for a smooth and efficient project delivery.

	AVR.	MAI	JUIN
Sprints	SPRINT 1	SPRINT 2	SPRINT 3
Versions			
> IN-67 Setup project and database structure			
> IN-73 Create Project Guidelines and Initial Manage...			
> IN-2 MINTER Registration and Login			
> IN-7 Profile Creation and Customization			
> IN-6 MINTER Privacy/Security Settings			
> IN-8 Social Features (single MINTER)			
> IN-9 Social Features (multi MINTERS aka TEA BAG)			
> IN-13 Original Content Management TERMINÉ(E)			
> IN-15 Admin Dashboard			
> IN-10 Search			
> IN-11 Notifications			
> IN-12 Cryptocurrency Integration			
> IN-14 NFT Trading and Marketplace			

Durability of chosen solutions

The project is built using Next.js which is a highly popular React framework.

Next.js offers features that enhance the application's performance and maintainability throughout the entire development process:

1. Server-Side Rendering (SSR): This improves the application's performance and SEO by rendering pages on server before sending them to the client.

2. Static Site Generation (SSG): Next.js also allows for the pre-rendering of pages at build time which can improve load times and performance.

3. API Routes: These provide a way to create a backend functionality, facilitating integration between the frontend and the backend (also makes the CI easier to develop since only one build is necessary).

The project also utilizes Auth.js, Prisma and Tanstack Query (these are the main dependencies but there are others as well).

Why these choices ensure durability?

- **Scalability:** The chosen technologies are designed to handle high loads and can easily scale as the user base grows. For example, an API function deployed on Vercel is easily scalable using the Vercel CLI or UI. The application is then ready for the future with possibility an increased amount of users.
- **Security:** These robust frameworks helps the developer to implement the best security practices, protecting user data.
- **Active maintenance:** All the frameworks used are actively maintained by their communities/enterprises ensuring that they stay up-to-date with the latest development trends.

Quality processes

We adhere to strict quality processes to ensure that our codebase remains robust and maintainable. Every pull request (PR) to the `develop` branch is thoroughly reviewed by team members before being merged. This review process helps maintain code quality and consistency, catching potential issues early.

We follow a structured release cycle with one release per month to the `main` branch. This regular release schedule ensures that new features and improvements are rolled out in a controlled and predictable manner.

Our Continuous Integration (CI) pipeline plays a critical role in maintaining high-quality standards. It runs several checks to verify the quality of the code, including:

- **ESLint Linter:** Ensures that the code adheres to predefined coding standards and best practices by detecting code errors, style issues, and potential bugs.
- **Playwright End-to-End Tests:** These tests verify that the application functions correctly from a user perspective by simulating user interactions and checking the outcomes.
- **Jest Unit Tests:** These tests ensure that individual units of code function as expected, providing a reliable basis for the overall application stability.

Additionally, we maintain a dedicated Discord channel where we document and share all best practices to be followed by the team. This channel serves as a centralized repository of guidelines and is instrumental in promoting uniform coding standards across the team.

We are committed to continuous improvement in our development process. We regularly review and refine our coding practices, incorporating feedback and new learnings to enhance the quality and efficiency of our codebase. This iterative approach ensures that we stay ahead of potential issues and consistently deliver high-quality software.

By combining code reviews, a structured release schedule, automated quality checks, and a culture of continuous improvement, we ensure that our project remains reliable, secure, and performant.